Exp 12. End-End Communication at Transport layer

Implement echo client server using TCP/UDP sockets

Aim :

TO implement an Echo client-server application using TCP socket and verify communication between the client and server

Steps :

1) Create a TCP echo server using server socket and socket
2) Assign a port number to the server
3) Compile and execute the TCP server program
5) Connect the client to the server using the address and port numbers
6) Enter a message through the client
7) Display the echoed message at the client
8) Enter bye to terminate the communication

Result : Thus, the TCP echo-client-server application was successfully implemented and the client successfully received the echoed message from the server

Program

```
Import java.io.*;
Import java.net.*;
Public class TCPEchoClient{
    Public static void main(String[]args) throws IOException{
    String host = "Local host";
    Int port = 5000;
    Socket socket = new Socket(host, port);
    BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
    PrintWriter out = new PrintWriter(
            Socket.getOutputStream(), true);
    System.out.println("connected to TCPEchoServer");
    System.out.println("Type a message (bye to exit)");
    String message;
    while((message = keyboard.readline()) != null){
            out.println(message);
            if(message.equals Ignore Case("bye")){
                break;
            }
    }
    Socket.close();
```

Ex.No:2b) Echo client - Server using UDP

Aim: To Implement as Echo client - server application using UDP socket and verify communication between the client and server

Procedure:

1) Create a UDP Echo server using DatagramSocket and Datagram packet
2) Assign a port number to the UDP server
3) Compile and execute the UDP server program
4) Create a UDP echo client using Datagram Socket and datagram packet
5) Specify the server address and port number
6) Enter a message through the client
7) The client sends the message as a UDP datagram to the server
8) The server receives the datagram and sends the echoed message back
9) Display the received echoed message at the client.
10) Enter data to terminate the communication

Result: Thus, the UDP Echo client - server application was successfully implemented and the client successfully received the echoed message from the server

Program:

```
import java.net.*;
Public class UDPEchoServer{
  Public static void main (String[] args) throws IOExcep
    int port=6000;
    DatagramSocket socket=
        new DatagramSocket(port);
    byte[] buffer = new byte[1024];
    System.out.println(
        "UDP Echo Server started on port "+port);
    while (true){
        DatagramPacket packet=
            new DatagramPacket(buffer, buffer.leng
        socket.receive(packet);
        String message=
            new String(
                packet.getData(),
                0,
                packet.getLength());
        System.out.println(
            "Received: "+message);
        byte[] responseData=
            ("Echo: "+message).getBytes();
        socket.send(response);
    System.out.println("UDP server stopped.");
```

e 12 c)

Chat client-server using TCP/UDP socket

Aim: To implement a chat-server application using sockets and enable communication between multiple clients through a server.

Procedure:

1) Create a chat server program using socket programming
2) Start & run the chat server
3) Create the chat client program and connect it to the server
4) Run two or more client programs in seperate terminals
5) Enter a name of each client
6) Send message from one client to the server
7) The server broadcasts the message to through the server
8) Continue that communication between the clients
9) Enter quit to terminate a client connection

Result: Thus, the chat client-server application was successfully implemented, and message was successfully exchanged between multiple clients through the server.

Program:

```
import java.io.*;
import java.net.*;
public class chatclient{
    public static void main (String [] args)
            throws IOException {
        String host = "localhost";
        int port = 5000;
        Socket socket =
                new Socket (host, port);
        BufferedReader in =
                new BufferedReader (
                        new InputStreamReader (
                                socket.getInputStream()));
        PrintWriter out =
                new PrintWriter (
                        socket.getOutputStream(),
                        true);
        try {String message;
            while ((message =
                        in.readLine()) != null){
                System.out.println(message);
            }
    }};
    listner.setDaemon(true);
    listener.start();
    String message;
    while ((message = keyboard.readline()) != null){
            out.println(message);
    }}
    socket.close();
    System.out.println(
            "Chat client closed.");
```